InboxIQ

Smart Email Classifier with Reminder System

Project Report

Submitted by: Kartiki Sinha
GitHub: github.com/kartiki0506/InboxIQ
Registration Number: 25BAS10020

1. Abstract

A Flask-based web application called InboxIQ works with Gmail to automatically categorise incoming emails into useful groups like Academic, Event, Internship, Spam, and General. The system analyses email subjects and body content using keyword-based natural language processing, then displays the findings on a web dashboard that can be filtered. InboxIQ also offers a reminder feature that lets users set up desktop alerts for critical emails at a predetermined time. The project shows how web development, API integration, and lightweight text classification methods can be used in real-world scenarios.

2. Introduction

Email continues to be one of the most popular tools for academic and professional correspondence despite the exponential growth of digital communication. But keeping an organised inbox has grown more difficult. In the midst of promotions, event announcements, and spam, both professionals and students find it difficult to recognise and prioritise important emails.

InboxIQ was created as a lightweight, intelligent email management solution to tackle this problem. The program automatically classifies emails using a rule-based classification engine and the Gmail API, freeing users to concentrate on what really matters. By enabling timely desktop notifications for follow-ups, the integrated reminder system further boosts productivity.

The key goals of this project are:

- Automate email classification using subject and body content analysis.
- Provide a clean, filterable web-based dashboard for browsing categorized emails.
- Enable users to set time-based reminders with desktop notifications.
- Ensure secure API access by requiring individual OAuth credentials.

3. Problem Statement

Every day, a large number of messages are sent to modern email inboxes on a variety of subjects, including commercial offers, event invitations, academic notifications, internship opportunities, and pointless spam. Sorting through these emails by hand takes a lot of time and is prone to mistakes.

Without the need for sophisticated machine learning infrastructure, an automated system that can intelligently classify emails at the moment of viewing, display them in a structured dashboard, and enable users to be reminded about crucial emails at a predetermined time is needed.

4. Objectives

- Connect to Gmail using OAuth 2.0 via the Gmail API.
- Fetch the latest 15 emails from the user's inbox.
- Classify each email into one of five categories: Internship, Event, Academic, Spam, or General.
- Display emails on a filterable web dashboard built with Flask and HTML/CSS.
- Allow users to select an email and set a reminder for a specific time.
- Deliver desktop push notifications using the Plyer library when a reminder is due.

- Persist reminders across sessions by saving them to a local text file.

5. Technologies Used

Technology	Version / Type	Purpose
Python	3.x	Core programming language for the backend logic
Flask	Latest	Lightweight web framework to serve routes and templates
Gmail API	v1 (REST)	OAuth 2.0-based access to user's Gmail account
google-auth-oauthlib	Latest	Handles OAuth authentication flow with Google
Plyer	Latest	Cross-platform desktop notifications for reminders
HTML & CSS	HTML5 / CSS3	Frontend templates for cover, dashboard, and reminder pages
Jinja2	Built-in Flask	Template engine for rendering dynamic HTML content
Threading	Python stdlib	Background thread management for reminder scheduling
Base64	Python stdlib	Decoding email body content from Gmail API response

6. System Architecture

InboxIQ follows a three-tier architecture comprising the presentation layer (HTML templates), the application layer (Flask routes and Python logic), and the data/service layer (Gmail API and local file storage).

6.1 Presentation Layer

Three HTML templates form the user interface:

- cover.html – Landing page with navigation to the dashboard.
- dashboard.html – Main view displaying categorized emails with filter options.
- reminder.html – Form page for selecting an email and scheduling a reminder.

6.2 Application Layer

The Flask application (app.py) manages all business logic:

- Authentication module using InstalledAppFlow for Gmail OAuth.
- Email fetching module using the Gmail API messages.list and messages.get endpoints.
- Classification engine using keyword matching on combined subject and body text.
- Reminder manager using Python's threading module to schedule notifications.

6.3 Data/Service Layer

- Gmail API – Source of email data via secure OAuth 2.0 authentication.
- reminders.txt – Flat file used to persist reminder data across application restarts.
- credentials.json – User-supplied Google OAuth credentials (not stored in the repository).

7. Project Structure

File / Folder	Description
app.py	Main Flask application containing all routes, classification logic, Gmail integration, and reminder management
requirements.txt	Python dependency list for setting up the project environment
templates/cover.html	Landing/welcome page of the application
templates/dashboard.html	Email dashboard with category filtering
templates/reminder.html	Reminder scheduling form
screenshots/	Directory containing UI screenshots of the application
.gitignore	Excludes sensitive files such as credentials.json and token files
reminders.txt	Auto-generated at runtime to persist reminder entries

8. Core Modules and Functionality

8.1 Gmail Authentication

Authentication is implemented using the google-auth-oauthlib library. On first run, the application opens a browser window for the user to authorize Gmail read access (SCOPES: gmail.readonly). The resulting credentials are cached in the global service variable to avoid re-authentication on subsequent requests within the same session.

8.2 Email Fetching

The get_emails() function retrieves the 15 most recent messages from the authenticated inbox. For each message, it extracts the Subject and From headers, then decodes the plain-text body using Base64 URL-safe decoding. Both multipart and single-part message structures are handled.

8.3 Email Classification Engine

The classify_email() function applies keyword-based classification by scanning the combined subject and body text (converted to lowercase). Classification rules are evaluated in order of priority:

- Internship – keywords: internship, apply, career, job
- Event – keywords: event, workshop, seminar, webinar
- Academic – keywords: gate, exam, result, program, course
- Spam – keywords: win, offer, discount, sale
- General – default fallback for unmatched emails

8.4 Dashboard and Filtering

The /dashboard route passes all fetched and classified emails to the dashboard.html template. An optional filter query parameter allows users to view emails belonging to a specific category only. The Jinja2 template engine renders the email list dynamically.

8.5 Reminder System

The /reminder route handles both displaying available emails and processing reminder form submissions. On form submission, the application:

- Parses the user-provided hour, minute, and AM/PM values.
- Converts the time to a 24-hour format and calculates the delay in seconds from the current time.
- Saves the reminder entry to reminders.txt in the format email_text|seconds.
- Spawns a background daemon thread that sleeps for the calculated duration and then calls Plyer's notification.notify() to deliver a desktop alert.
- On application restart, load_reminders() reads saved entries and re-spawns the corresponding threads.

9. Setup and Installation

9.1 Prerequisites

- Python 3.x installed on the system.
- A Google account with Gmail access.
- A Google Cloud project with the Gmail API enabled.
- OAuth 2.0 credentials (Desktop App type) downloaded as credentials.json.

9.2 Installation Steps

1. **Clone the repository:** `git clone https://github.com/kartiki0506/InboxIQ.git`
2. **Install dependencies:** `pip install -r requirements.txt`
3. Place credentials.json in the root folder of the project.
4. **Run the app:** `python app.py`
5. **Open browser at:** `http://127.0.0.1:5000/`

10. Security Considerations

- OAuth 2.0 is used for authentication, ensuring that credentials are never stored in plaintext within the app.
- credentials.json is excluded from the repository via .gitignore to prevent accidental exposure.
- Each user must generate their own API credentials, maintaining independent and secure access.
- The application requests only the gmail.readonly scope, limiting access to read-only operations.
- No email content is stored on a server; all processing happens locally in memory.

11. Limitations

- The classifier uses rule-based keyword matching, which may misclassify emails with ambiguous or overlapping keywords.
- Only the 15 most recent emails are fetched per session, limiting the breadth of classification.
- Reminders are stored in a plain text file, which is not robust against corruption or concurrent access.

- If the reminder time has already passed today, the application schedules it for the next calendar day (basic handling).
- Desktop notifications depend on the Plyer library and may behave differently across operating systems.
- The app runs in debug mode by default and is not configured for production deployment.

12. Future Improvements

- Machine Learning Classification – Replace rule-based matching with a trained Naive Bayes or BERT-based model for higher accuracy.
- Email Analytics Dashboard – Add charts and statistics showing email volume by category over time.
- Search Functionality – Enable full-text search across all fetched emails.
- Calendar Integration – Sync reminders with Google Calendar for a unified scheduling experience.
- Dark Mode UI – Provide a dark theme option in the web dashboard.
- Pagination – Support loading more than 15 emails with infinite scroll or page navigation.
- Database Storage – Replace reminders.txt with a SQLite or PostgreSQL database for reliability.
- Email Reply / Action – Add the ability to reply to or archive emails directly from the dashboard.

13. Conclusion

InboxIQ successfully demonstrates how a lightweight, full-stack web application can solve a real-world productivity problem. By integrating with Gmail's official API, applying a keyword-based classification engine, and providing a reminder system with desktop notifications, the project delivers a functional and intuitive email management tool.

The application is built entirely with open-source technologies and follows a clean modular architecture, making it straightforward to extend with more advanced features such as machine learning-based classification or calendar synchronization in future iterations.

This project strengthened skills in Python web development, REST API integration, OAuth 2.0 authentication flows, multithreading, and frontend templating — making it a well-rounded demonstration of practical software engineering.

14. References

- Google Gmail API Documentation – https://developers.google.com/gmail/api
- Flask Framework Documentation – https://flask.palletsprojects.com/
- Google Auth OAuthLib – https://google-auth-oauthlib.readthedocs.io/
- Plyer Notifications – https://plyer.readthedocs.io/
- Python Threading Documentation – https://docs.python.org/3/library/threading.html
- InboxIQ GitHub Repository – https://github.com/kartiki0506/InboxIQ